

Numerical Solution for the Shallow Water Equations in 2D

Ethan Chow

May 2025

1 Abstract

The Shallow Water Equations (SWE) have many important applications in determining the outcomes of flooding, river flow, and other hydraulic uses. In this project, a numerical method derived from the Lax-Friedrich scheme was used in Python to solve the equations and determine the fluid surface height and horizontal velocity over time. The results of this method were successful in modeling multiple cases of the SWE, including a sloped sinusoidal and rocky bottom, as well as a rain simulation. A drawback to this method is the consequential dampening that results from applying this method to moving waves, which is further amplified when modeling in 2-D. In this analysis, the numerical method, results, and stability conditions will be discussed.

2 Introduction

The SWE are hyperbolic partial differential equations (PDE) that are used to describe the motion of fluids that were derived by Adhémar Jean Claude Barré de Saint-Venant[1]. They describe fluids with constant density, shallow depth, and in hydrostatic balance. The shallow water assumption means that the horizontal flow is significantly greater than the layer depth, making these equations valid[6]. These equations have many applications in physics and engineering, providing information on river flow, storm surges, and flooding. Finite volume methods are accepted in literature as an accurate tool for the SWE, and were originally used for gas dynamics[2]. Using finite volume methods, applications to the shallow water equations such as realistic flood modeling have been done[3]. The goal of this project was to analyze water in a shallow-water setting with varying bottom topography and initial conditions. The non-conservative form of the shallow water equations is derived from conservation of mass and

momentum equations. This results in three equations which are as follows[4]:

$$\frac{\partial \eta}{\partial t} + \frac{\partial}{\partial x} [hu] + \frac{\partial}{\partial y} [hv] = 0, \quad (1)$$

$$\frac{\partial u}{\partial t} + u \frac{\partial u}{\partial x} + v \frac{\partial u}{\partial y} = -g \frac{\partial \eta}{\partial x} - \gamma u, \quad (2)$$

$$\frac{\partial v}{\partial t} + u \frac{\partial v}{\partial x} + v \frac{\partial v}{\partial y} = -g \frac{\partial \eta}{\partial y} - \gamma v. \quad (3)$$

Where η represents the surface height, h the depth of the water, u the velocity in x , and v the velocity in y , and g the gravitational constant. γ represents the coriolis parameter which in this case is ignored.

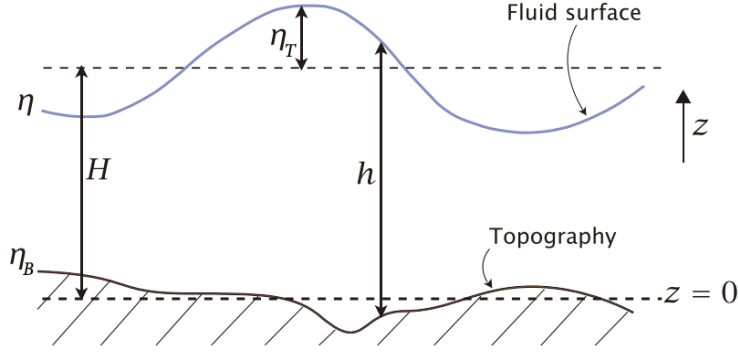


Figure 1: Visual of the various variables in the SWE. Note: In this work I have used η_b interchangeably with b

3 Methodology

3.1 Simplified SWE

From equations (2) and (3), the coriolis term is ignored. Additionally, non-linear terms are ignored as small amplitude waves are what is being analyzed. This results in the following[6]:

$$\frac{\partial u}{\partial t} = -g \frac{\partial \eta}{\partial x}, \quad (4)$$

$$\frac{\partial v}{\partial t} = -g \frac{\partial \eta}{\partial y}, \quad (5)$$

$$\frac{\partial \eta}{\partial t} = -h(x, y) \left(\frac{\partial u}{\partial x} + \frac{\partial v}{\partial y} \right). \quad (6)$$

3.2 Numerical Method

The Lax-Friedrich numerical method for PDEs that this method is based upon is as follows[5]: For a hyperbolic conservation law of the form:

$$\frac{\partial u}{\partial t} + \frac{\partial f(u)}{\partial x} = 0,$$

the Lax-Friedrichs finite difference scheme is given by:

$$u_i^{n+1} = \frac{1}{2} (u_{i+1}^n + u_{i-1}^n) - \frac{\Delta t}{2\Delta x} (f(u_{i+1}^n) - f(u_{i-1}^n)),$$

where u_i^n is the numerical approximation of $u(x_i, t^n)$, Δx is the spatial step size, Δt is the time step size, and $f(u)$ is the flux function.

Taking equations (4), (5), and (6) in 1-D, (all v terms are 0), and then using a forward in time, centered in space (FTCS) scheme for u and the Lax-Friedrich scheme for η results in the following:

u term update:

$$u_i^{n+1} = u_i^n - \Delta t \cdot g \cdot \frac{\eta_{i+1}^n - \eta_{i-1}^n}{2\Delta x}$$

η term update:

$$\begin{aligned} h_j^n &= \eta_j^n - b_j \\ \eta_j^{n+1} &= -h_j^n \cdot \frac{u_{j+1}^{n+1} - u_{j-1}^{n+1}}{2\Delta x} \cdot \Delta t + \frac{\eta_{j+1}^n + \eta_{j-1}^n}{2} \end{aligned}$$

Where b is the bottom topography and the entire u vector is updated first hence the two indices i and j , however the loops have the same duration. This allows for the use of u_{j+1}^{n+1} & u_{j-1}^{n+1} in the η equation. It is possible to start with either η or u , however η is the variable being initialized therefore it is logical to start with the u update.

The FTCS method works well for u because multiplying by g , a constant, keeps stability in u . However h is not a constant, therefore this method is too unstable for η , therefore the Lax-Friedrich scheme was necessary. The issue with the Lax-Friedrich method is that for these asymmetrical waves, the η_{j+1}^n step has a significant decrease, which inadvertently results in large damping.

The solution to this is to take the Lax-Friedrich scheme further, and to take the mean of η_{j+1}^n and η_j^n as well as the mean of η_{j-1}^n and η_j^n , and average these two quantities. This is in theory taking the mean of a predicted half step forward and a predicted half step backward, which consolidates the values more towards the current step while keeping numerical stability. This scheme for 1-D is shown below:

$$\eta_j^{n+1} = -h_j^n \cdot \frac{u_{j+1}^{n+1} - u_{j-1}^{n+1}}{2\Delta x} \cdot \Delta t + \frac{1}{2} \left(\frac{\eta_{j+1}^n + \eta_j^n}{2} + \frac{\eta_{j-1}^n + \eta_j^n}{2} \right)$$

And the final full numerical method in 2-D is:

2-D velocity updates:

$$\mathbf{v}_{:,i}^{n+1} = \mathbf{v}_{:,i}^n - \Delta t \cdot g \cdot \frac{\eta_{:,i+1}^n - \eta_{:,i-1}^n}{2\Delta y}, \quad (7)$$

$$\mathbf{u}_{i,:}^{n+1} = \mathbf{u}_{i,:}^n - \Delta t \cdot g \cdot \frac{\eta_{i+1,:}^n - \eta_{i-1,:}^n}{2\Delta x} \quad (8)$$

2-D η update: Let $h_{j,l}^n = \eta_{j,l}^n - b_{j,l}$. Then:

$$\eta_{j,l}^{n+1} = -h_{j,l}^n \cdot \frac{u_{j+1,l}^{n+1} - u_{j-1,l}^{n+1}}{2\Delta x} \cdot \Delta t - h_{j,l}^n \cdot \frac{v_{j,l+1}^{n+1} - v_{j,l-1}^{n+1}}{2\Delta y} \cdot \Delta t \quad (9)$$

$$+ \frac{1}{2} \left[\frac{1}{2} (\eta_{j+1,l}^n + \eta_{j,l}^n) + \frac{1}{2} (\eta_{j-1,l}^n + \eta_{j,l}^n) \right] \quad (10)$$

$$+ \frac{1}{2} \left[\frac{1}{2} (\eta_{j,l+1}^n + \eta_{j,l}^n) + \frac{1}{2} (\eta_{j,l-1}^n + \eta_{j,l}^n) \right] \quad (11)$$

Where index i is able to be used for both directions in (7) and (8) since dy and dx are the same size and indices j and l are used in (9), (10), and (11) to avoid confusion with (7) and (8). It is important to note that while this method is an improvement on the Lax-Friedrich method, it still has inadvertent damping, which is the tradeoff for numerical stability.

3.3 Numerical Stability Constraints

The Courant-Friedrichs-Lewy (CFL) Condition is a condition that determines the maximum size of the timestep. It is used for hyperbolic PDEs to ensure numerical stability. In simple terms, it means that the next time step must be slower than the speed of the wave[7]. Violating this conditions causes the numerical simulation to "blow up". The Courant-Friedrichs-Lewy (CFL) condition for stability of the explicit scheme is:

$$\Delta t \leq \frac{\text{CFL} \cdot \min(\Delta x, \Delta y)}{\sqrt{g(H + \eta_{\max})}} \quad (12)$$

Where H is the mean water depth, and CFL is a constant (in this case < 1) chosen for numerical stability.

4 Results

This method was successful in modeling the SWE in both 1-D and 2-D. Six cases were studied all on the same size xy -grid, with x (and y for 2-D cases) going from -10 to 10. They were initialized with a gaussian bump of the form

$$\eta(x, y, t = 0) = a \cdot e^{-c(x^2+y^2)} + H$$

Where a and c are constants. These cases are as follows:

1. 1-D flat bottom, $b(x) = 0.1$
2. 1-D sloped sinusoidal bottom, $b(x) = \frac{1}{20} \cdot \sin(2x) + \frac{1}{26}x + 0.5$
3. 1-D simulated rock, $b(x) = \frac{1}{1.3} \cdot e^{-0.4(x+5)^2}$
4. 2-D sloped sinusoidal bottom, $b(x, y) = 0.1 \cdot \sin(x) + \frac{1}{26}x + 0.5$
5. 2-D simulated rocky bottom, $b(x, y) = 0.6 \cdot e^{-0.05((x+2)^2+(y+5)^2)} + 0.3 \cdot e^{-0.1((x+5)^2+(y+5)^2)} + 0.3 \cdot e^{-0.1((x+8)^2+(y+9)^2)} + 0.1 \cdot \sin(x) + \frac{1}{300}x^2$
6. 2-D simulated rain, $b(x, y) = 0.1$ *Will mention rain simulation methodology later

4.1 1-D Flat Bottom

The 1-D flat bottom case was used to develop the numerical method and to analyze shallow water waves with nothing else affecting it. The waves move away from the initial gaussian bump with equal velocity and each wave is asymmetrical with a sharper incline in the direction it is traveling. The waves dampen over time and reflect on the boundaries as expected. View the animation [here](#).

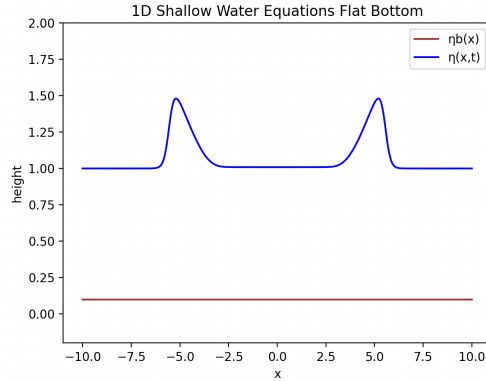


Figure 2: Flat Bottom 1-D Case: $\eta_b(x)$ is synonymous to $b(x)$. $\eta(x, t)$ is the surface height, which is equal to $h(x) + b(x)$

4.2 1-D Sloped Bottom

The 1-D sloped bottom case was inspired by a shoreline, however due to boundary conditions it is not representative of a beach. This case represents a closed system with a sloped bottom topography. The result is that the wave traveling towards the shorter depth side has less velocity and amplitude. and the two waves interfere closer to the shallow side. After the initial interference, the

waves next collide again in the middle, showing that the motion is periodic. View the animation [here](#).

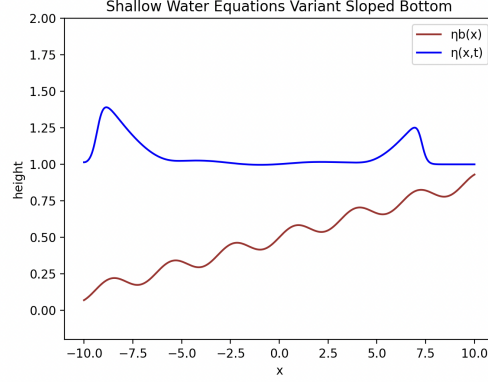


Figure 3: 1-D Sloped Bottom Case

4.3 1-D Rock Bottom

In this case, a "rock" was created using a gaussian bump. The wave loses amplitude and velocity when passing over the rock, and regains its velocity after passing. This also causes disturbance in the surface, however for the second wave collision they still interfere in the middle. View the animation [here](#).

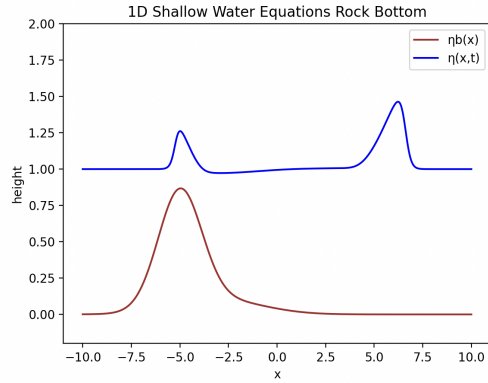


Figure 4: Gaussian Bump Simulated Rock Bottom Case

4.4 2-D Sloped Bottom

The simulation in 2-D required more computational power, as for the η calculation two loops were required. In this simulation, a sloped bottom similar to

case 2 was used. This showed similar results along x and showed symmetry along y which is intuitive. As shown in Figure 4, the shallow side waves have less amplitude, as well as less velocity which can be seen in the animation. View the animation [here](#).

2D Shallow Water Equations with Variant Slope Bottom

Zoomed In (Shrunken Z Axis)

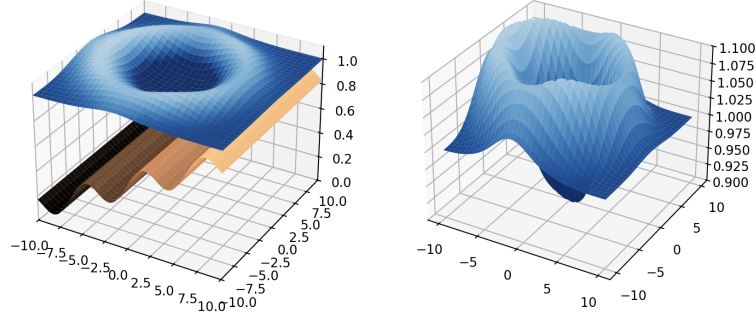
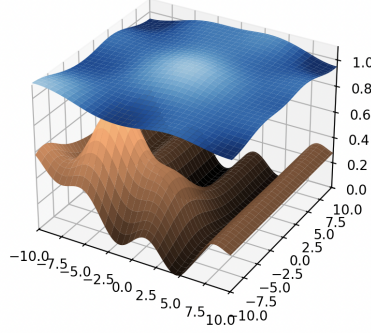


Figure 5: 2-D Sloped Bottom Case, The left plot includes the bottom in the model and the right plot is zoomed in over z to better visualize the wave

4.5 2-D Rocky Bottom

This case shows a closed system with a heavily varying bottom topography, as can be seen by the $b(x)$ initialization. This similarly shows less amplitude and velocity over shallower regions, however the variant bottom causes a disturbed, unsymmetrical surface. View the animation [here](#).

2D Shallow Water Equations with Rocky Bottom



Zoomed In (Shrunken Z Axis)

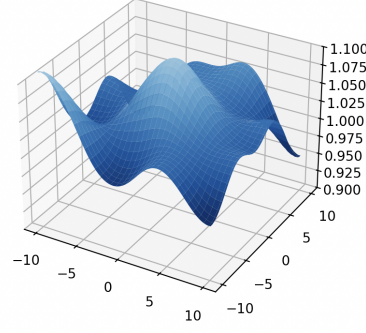


Figure 6: 2-D Rocky Bottom Case

4.6 2-D Rain Simulation

In this case, rain was simulated to create a constant addition of disturbance to the closed system. This was done by adding randomized "rain drops" to η . The parameters for this randomness were creating 0 to 3 rain drops at every 120 time steps ($\approx 0.25s$).

If $t \bmod 120 = 0$, then

$$\eta(x, y, t) = \eta(x, y, t) + \sum_{i=1}^3 \left(\frac{r_i}{4} \cdot e^{-5[(x+a_i)^2 + (y+b_i)^2]} \right)$$

Where r_i is randomly 0 or 1, and a_i and b_i are random integers from -9 to 9. While this is a very simple way to simulate rain, it is more to show the motion of the surface of a shallow fluid with constant outside disturbance. A intuitive but interesting result of this simulation was the mass of the fluid increasing over time, which is visible in the simulation. View the animation [here](#).

2D Shallow Water Equations with Simulated Rain

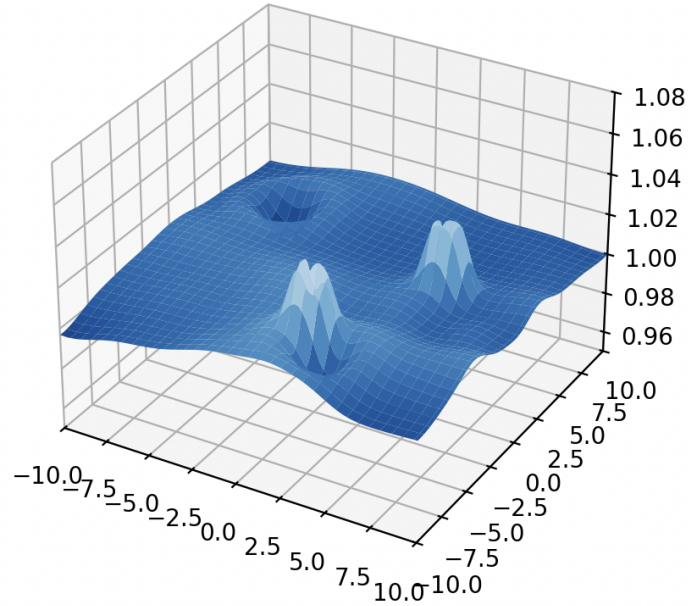


Figure 7: 2-D Rain Simulation

5 Conclusion

In conclusion, this numerical method was successful in modeling fluids in 2-D with various bottom topographies as well as a constant addition of mass. The downside to this method is the unavoidable damping that occurs from using forward spacial steps with asymmetrical waves. Amplifying the waves is also not possible as it will result in nonphysical results such as energy of the system not being conserved. Future work would include attempting quarter steps to determine if numerical stability would remain or trying additional case studies. Additional future work would be adding phase shifts for bottom topography that exceeds the fluid height.

References

- [1] Max Zhao, *2-Dimensional Shallow Water Equations Solver using FDM*, Dec. 2, 2022

- [2] García-Navarro, P., Murillo, J., Fernández-Pato, J. et al. *The shallow water equations and their application to realistic cases. Environ Fluid Mech* 19, 1235–1252 (2019). <https://doi.org/10.1007/s10652-018-09657-7>
- [3] H. Ali, P-Y. Lagree, J-M. Fullana *Application of the Shallow Water Equation to Real Flooding Case*
- [4] Harvey Segur & Hiroki Yamamoto, *Lecture 8: The Shallow-Water Equations*, June 18, 2009
- [5] L.F. Shampine, Southern Methodist University, *Two-step Lax-Friedrichs Method*
- [6] Vallis GK. Shallow Water Systems. In: *Atmospheric and Oceanic Fluid Dynamics: Fundamentals and Large-Scale Circulation*. Cambridge University Press; 2017:105-142.
- [7] Carlos A. de Moura & Carlos S. Kubrusly *The Courant–Friedrichs–Lewy (CFL) Condition: 80 Years After its Discovery* 2013